If you ever need to see the full backtrace, you still can; just pass the --backtrace or -b flag to RSpec.

Take some time to read through the comments inside spec_helper.rb. They explain how the new configuration options will affect your tests. Once you're done, you'll be ready to write some unit tests.

Sketching the Behavior

The behavior you're testing breaks down into a couple of broad categories. You'll want to see what happens when an API call succeeds and when it fails.

Use Unit Specs for Edge Cases



Unit tests' speed and simplicity make them the perfect place to test all your conditional branches and edge cases. Exhaustively covering all the cases in a slower integration or acceptance test tends to be too inefficient.

Start by sketching out the success case. Create a new file called spec/unit/app/api_spec.rb with the following contents:

05-unit-specs/02/expense_tracker/spec/unit/app/api_spec.rb

```
require_relative '../../../app/api'

module ExpenseTracker
  RSpec.describe API do
    describe 'POST /expenses' do
      context 'when the expense is successfully recorded' do
        it 'returns the expense id'
        it 'responds with a 200 (OK)'
      end

      # ... next context will go here...
    end
  end
end
```

The context block groups these two success-related specs together and allows you to share common setup code between them.

Now, add a second context. This one will handle the failure case:

05-unit-specs/02/expense_tracker/spec/unit/app/api_spec.rb

```
context 'when the expense fails validation' do
  it 'returns an error message'
  it 'responds with a 422 (Unprocessable entity)'
end
```

This is enough content to get started. Run your suite now, and RSpec will report these test cases as pending: